

SEDust: User Manual

Abstract

This manual describes the model-agnostic dust library under SEDust/sed/. It computes the infrared emission spectrum *and* the transport optics (extinction, scattering, asymmetry) of a dust mixture for an arbitrary local radiation field, and is intended to be linked into a Fortran 3D radiative-transfer (RT) code. It handles several dust models as peers — the HD23 *astrodust*+PAH model, the Draine & Li (2007) silicate+carbonaceous model, the Zubko, Dwek & Arendt (2004) BARE-GR-S model, and arbitrary file-defined models — through one object type, one emission call, and one extinction call. The numerically validated stochastic-heating solver is reused unchanged; the library is an additive layer on top of it.

This version is the **scalar** branch (v1.00): unpolarized cross sections and unpolarized emission. There is no polarized-transfer API here — no dichroic extinction matrix, no aligned-grain scattering matrix. Use v1.20 for those.

Contents

1. Overview	2
2. Building the library	3
3. Models and builders	3
4. Computing emission	5
4.1 Single-cell exact solve	5
4.2 Solver choice (m%stoch_method)	5
4.3 Equilibrium-temperature options	5
4.4 Precomputed table + interpolation	6
4.5 Optional error status	6
5. Computing extinction	7
5.1 What the call returns, and where it comes from	8
5.2 Dust mass per H, and mass opacity	9
6. The extreme-ultraviolet extension	10
6.1 Where the optics come from in the extended band	10
6.2 The two routes, and what the cheap one costs	11
6.3 The single-photon ceiling comes from the field	13
6.4 What else the extension does, and does not, change	15
6.5 How far each model can be checked	15
7. Standalone programs	16
7.1 calc_kext.x — size-integrated transport optics	16
7.2 main_astrodust.x — the astrodust production SED	17

7.3	main_dl07.x — the DL07 reference SED	17
7.4	main_zubko.x — the Zubko/ZDA SED, and the wide-grid regression	17
7.5	make_enthalpy.x — enthalpy tables (diagnostic)	18
8.	The model object and accessors	18
9.	The generic file loader	19
10.	Linking into a 3D RT code	19
11.	Units and conventions	20
12.	Data files	20
12.1	Transport-optics products	20

1. Overview

A *dust model* is represented by a single derived type, `dust_model_t`. You build it once (per RT run), then call `dust_emission` once per RT cell with that cell's local mean intensity:

```

use dust_lib
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:)

call build_astrodust(m, qtab, sizedist, 200, 2.7_wp, 5.0e3_wp) ! once
allocate(J_lam(m%NLAM), lamI_total(m%NLAM))
do icell = 1, ncells
  ! ... assemble J_lam(:) (mean intensity on the grid m%lam) for this cell ...
  call dust_emission(m, J_lam, lamI_total) ! per cell
  ! ... lamI_total(:) is lambda*I_lambda / N_H for the cell ...
end do

```

Design. The library wraps the existing, FIR-validated solver core (`sed_grain_loop`, `calc_Teq`, `calc_P`) without modifying it. A model is a set of grain *populations* (one per species and charge state), each carrying its own size distribution, absorption cross sections, enthalpy, and Planck-integral tables. `dust_emission` loops the populations through the solver and sums their emission into named output *channels*.

One active model at a time. The module-global grids (`lam`, `T_first`, ...) hold the *currently built* model's working set. Calling `build_*` makes that model active. This is sufficient for an RT run that uses a single dust model; to switch models, rebuild.

2. Building the library

Two archives, built in this order:

```
cd SEDust/tmatrix && make libtmatrix.a # T-matrix, needed by the EUV band
cd SEDust/sed && make libsedust.a # the library archive
```

libtmatrix.a must exist first: the astrodust extreme-ultraviolet band (§6.) computes its optics with the T-matrix, so every SEDust target that links sed_astrodust.f90 links it too. The sed Makefile stops with an instruction rather than a link error when it is missing.

The compiler is gfortran. The default flags are `-O3 -ffree-line-length-none -fopenmp` plus a warning set — deliberately *portable*, so that a libsedust.a built on one node runs on any other. A `-march=native` line is present but commented out; enabling it can change the last digits through FMA contraction. libsedust.a is a static archive of all library objects; the Fortran .mod files are written alongside it in SEDust/sed/. The Makefile declares .NOTPARALLEL, because every executable recipe compiles the shared module sources into the same directory and `make -j` would race on the .mod files.

3. Models and builders

Each builder fills a `dust_model_t` and defines the model’s output channels (Table 1).

builder	model	channels	notes
build_astrodust	HD23 astrodust+PAH	AD, PAH	$N_C=417$, D16 graphite
build_dl07	Draine & Li 2007	SIL, CARB	$N_C=470$, D03 graphite
build_zubko	Zubko (ZDA 2004) BARE-GR-S	PAH, GRA, SIL	neutral PAH only
build_from_files	file-defined	CH1, CH2, ...	generic loader

Table 1: Built-in model builders. Each sets the model’s own optics/abundance parameters internally.

Signatures (all optional arguments in brackets; every optional is a keyword argument in the order shown):

```
call build_astrodust(m, qtable_path, sizedist_path, NT, T_lo, T_hi &
    [, status] [, lam_min] [, astrodust_index_path] [, kext_path] &
    [, euv_tmatrix])
call build_dl07 (m, qtable_path, sizedist_path, sd_index, U, NT, T_lo, T_hi &
    [, status] [, lam_min] [, kext_path])
call build_zubko (m, config_path, data_dir, NT, T_lo, T_hi [, status] [, kext_path])
call build_from_files(m, descriptor_path, data_dir, NT, T_lo, T_hi [, status] [, kext_path])
```

- NT, T_lo, T_hi: number and range [K] of the internal temperature grid (typical 200, 2.7, 5000).
- sd_index (DL07): WD01 size-distribution index — 7 = MW $R_V=3.1$, $q_{\text{PAH}}=4.58\%$; 29/30/31 = LMC2; 32 = SMC.

- `U` (DL07): ISRF intensity scaling (the DL07 reference uses $U=1$).
- `data_dir` (Zubko / files): directory holding the data files, e.g. `data/zubko/`.
- `status` (optional, all builders): error report; see §4.5. When present, a missing or malformed input file is reported through it and the model is not built, so an RT host can recover.
- `lam_min` (optional, astrodust and DL07): shortest wavelength [μm] the model must cover, for a host that transports ionizing radiation. See §6.. **Omitting it reproduces the previous behavior exactly** — the grid, and every number derived from it, is unchanged.
- `astrodust_index_path` (optional, astrodust only): the dielectric function that supplies the optics below $0.0912\mu\text{m}$. It must be the file `qtable_path` was computed from — same porosity, iron fraction and axial ratio — or the model changes material at the seam. Omitted, it is the default that pairs with the shipped Q table:

```
data/dielectric/index_DH21Ad_P0.20_0.00_1.400 <->
tmatrix/output/q_astrodust_P0.20_Fe0.00_1.400.dat
```

- `kext_path` (optional, all builders): the precomputed size-integrated extinction table this model will serve through `dust_extinction` (§5.). The builder loads it, because the path is relative to `sed/` and a host is free to change directory once the model is built. Naming a file is a contract: if it cannot be read the *build fails*. Omitted, the model's default table (Table 2) is tried, and a default that is not there leaves the model with no extinction to serve — but the build succeeds, so a driver that only computes emission still runs.
- `euv_tmatrix` (optional, astrodust only, default `.true.`): how the extended band's optics are computed. `.true.` is the T-matrix on the same $b/a = 1.400$ oblate spheroid as the Q table; `.false.` substitutes the volume-equivalent-sphere Mie approximation, which is far cheaper and about 2% low in the geometric-optics limit. See §6.2. Ignored when `lam_min` asks for no extension.

builder	default <code>kext_path</code>
<code>build_astrodust</code>	<code>../data/kext_astrodust_MW_euv.dat</code>
<code>build_dl07</code>	<code>../data/kext_dl07_MW_euv.dat</code>
<code>build_zubko</code>	<code>../data/kext_zubko_BARE_GR_S.dat</code>
<code>build_from_files</code>	<code>none</code>

Table 2: Default extinction table of each builder. The EUV product is the default wherever the model has one, because its wavelength range *contains* the plain grid's: the two are the same size integral on the same optics, the extended one merely carried below $0.0912\mu\text{m}$, and their nodes coincide over the overlap. One default therefore serves a host that transports ionizing radiation and one that does not; it is the *narrower* non-EUV table that has to be named explicitly. The Zubko model has no EUV extension and needs none — its own optics table already reaches 1.24 keV.

Two routes to the Zubko model. `build_zubko` evaluates the ZDA log-polynomial size-distribution *formula* (coefficients read from `ZDA_BARE_GR_S_Config.dat`) on the optics-table radii; `build_from_files` with `data/zubko/zubko_descriptor.txt` instead interpolates the *tabulated* `SzDist` files onto the same radii. The two shapes agree to $\sim 10^{-5}$, but the distributed tables carry a normalization that sits below $Ag(a)$ of the config by 0.79% (PAH), 1.13% (graphite) and 0.41% (silicate), and they are sampled at only 24 / 62 / 63 radii, so interpolation near $a_{\max}$ adds a little more. Together this moves the size-integrated C_{ext} by up to 2.10% (median 1.02%). **The formula is the default and is faithful to the ZDA definition**; the tabulated route exists to reproduce the benchmark’s own size grid.

4. Computing emission

4.1 Single-cell exact solve

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status])
```

- `J_lam(m%NLAM)`: the local *mean intensity* on the grid `m%lam` [μm]. For a scaled Mathis ISRF use `call J_Mathis(U, m%lam, J_lam)` from module `radfield`.
- `lamI_total(m%NLAM)`: the summed SED, $\lambda I_{\lambda} / N_{\text{H}}$ [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1} \text{H}^{-1}$] — the HD23 convention.
- `lamI_chan(m%NLAM, m%n_channel)` (optional): the SED of each channel.
- `status` (optional): error report; see §4.5.

4.2 Solver choice (m%stoch_method)

The field `m%stoch_method` selects how each grain is heated (Table 3); `dust_emission` honors it.

<code>m%stoch_method</code>	method	note
'heuristic'	GD look-ahead T-window narrowing	default ; fastest, serial
'draine'	Guhathakurta & Draine (1989)	reference GD
'qm'	energy-space transition matrix (thermal-discrete)	most detailed
'equil'	single-grain equilibrium (no stochastic)	see §4.3

Table 3: Stochastic-heating solver options. The first three resolve the grain temperature *distribution* (PAH/NIR features); 'equil' forces equilibrium.

4.3 Equilibrium-temperature options

Two options replace the full stochastic solve when an equilibrium approximation is wanted:

(1) **Equilibrium temperature for each grain type and size.** Set `m%stoch_method = 'equil'` and call `dust_emission`. Every grain is placed at its own equilibrium temperature T_{eq} , computed from *that grain's* absorption cross section, and emits $C_{\text{abs}} B_{\lambda}(T_{\text{eq}})$. No stochastic excursions are computed.

(2) **A single equilibrium temperature for the whole model.**

```
call dust_emission_single_teq(m, J_lam, lamI_total [, Teq_out])
```

All dust shares one temperature, found from the type- and size-integrated (dn-weighted) total absorption cross section $C_{\text{abs}}^{\text{tot}}(\lambda) = \sum_{\text{pop}} \sum_a dn(a) C_{\text{abs}}(\lambda, a)$. The single T_{eq} satisfies $\int C_{\text{abs}}^{\text{tot}} J d\lambda = \int C_{\text{abs}}^{\text{tot}} B(T_{\text{eq}}) d\lambda$, and the emission is $C_{\text{abs}}^{\text{tot}} B_{\lambda}(T_{\text{eq}})$. The optional `Teq_out` returns that temperature.

Both options conserve energy (the emitted bolometric power equals the absorbed power); they differ from the stochastic solve only in the SED *shape* — a smooth modified blackbody rather than one carrying PAH/NIR features.

4.4 Precomputed table + interpolation

When the field in each cell is a fixed spectral *shape* scaled by an intensity U , precompute a table once and interpolate per cell:

```
type(dust_emis_table_t) :: tab
call dust_build_table(m, J_ref, U_grid, tab [, status]) ! once
call dust_emission_interp(tab, U, lamI_total [, lamI_chan] [, status]) ! per cell
```

`J_ref(m%NLAM)` is the reference field shape (at $U=1$) and `U_grid(:)` an ascending intensity grid. The table is built from exact solves, so it is exact at the grid points; interpolation is log–log in U . The stored emissivities are floored at 10^{-300} before the log, so a grid node whose true value is 0 comes back as 10^{-300} rather than exactly 0. Both calls take an optional final status (§4.5).

Caveat. Stochastic emission is a *nonlinear* functional of the full $J(\lambda)$, not of a scalar. The table is valid only when the cell field is $U \times (\text{fixed } J_{\text{ref}})$. For cells whose field *shape* departs from J_{ref} (e.g. hardened spectra near hot stars), use the cell-by-cell exact `dust_emission`.

4.5 Optional error status

The builders and the emission/table calls each accept an optional final status argument (integer, `intent(out)`). When it is present, an invalid model, argument, or input file is reported through it (`status = 0` on success, nonzero on failure) and the process keeps running, so a 3D RT host can recover; when it is omitted, the same condition prints a message and stops the run, which the CLI drivers rely on. For the builders the nonzero value only names the failing stage: the contract is simply `0 = built`, `nonzero = build failed`.

- `dust_emission`: 1 unknown `stoch_method`; 2 'qm' selected but a population's radii are unset.
- `dust_extinction`: 1 an output array is not of size `m%NLAM`; 2 no extinction table was loaded for this model; 3 `m%lam` reaches outside the loaded table's wavelength range.

- `dust_build_table`: 1 `size(J_ref) ≠ m%NLAM`; 2 `size(U_grid) < 2`; 3 `U_grid` not positive and strictly increasing.
- `dust_emission_interp`: 1 $U \leq 0$; 2 `size(lamI_total) ≠ tab%NLAM`; 3 `lamI_chan` present but not (`tab%NLAM`, `tab%n_channel`).
- `build_astrodust`: 1 Q -table load failed; 2 size-distribution load failed; 3 the astro dust dielectric function failed to load (raised only when `lam_min` asks for the EUV band); 4 `lam_min` is shorter than that dielectric function's own shortest wavelength ($1.00003 \times 10^{-4} \mu\text{m}$), which is refused rather than served with a refractive index frozen at the table boundary; 5 an explicitly named `kext_path` failed to load.
- `build_dl07`: 1 Q -table load failed; 2 size-distribution load failed; 4 `lam_min` is shorter than the D03 dielectric functions ($6.1992 \times 10^{-5} \mu\text{m}$ — the longer-reaching of astrosilicate and graphite), refused for the same reason (there is no code 3 on this path: this model needs no separate EUV dielectric function, because its optics are D03 Mie at every wavelength); 5 an explicitly named `kext_path` failed to load.
- `build_zubko`: 1 config read; 2 fewer than 3 components; 3 a component's optics read; 4 grid inconsistent; 5 a component's calorimetry read failed; 6 an explicitly named `kext_path` failed to load.
- `build_from_files`: 1 descriptor open; 2 too many pop: lines; 3 invalid channel; 4 no pop: lines; 5 optics read; 6 grid inconsistent; 7 size-distribution read; 8 calorimetry read failed; 9 an explicitly named `kext_path` failed to load.

A *default* extinction table that cannot be read is not an error on any of these paths: the model is built without one, and the failure surfaces later as `dust_extinction` status 2.

5. Computing extinction

`dust_extinction` is the extinction counterpart of `dust_emission`: a transfer code takes its opacity from the same model object, and on the same wavelength grid `m%lam`, as its emission.

```
call dust_extinction(m, Cext, Cabs, Csca [, gbar] [, albedo] [, status])
```

- `Cext`, `Cabs`, `Csca` (`m%NLAM`): extinction, absorption and scattering cross sections per H atom [cm^2/H], with $C_{\text{ext}} = C_{\text{abs}} + C_{\text{sca}}$.
- `gbar` (`m%NLAM`) (optional): the scattering-weighted asymmetry $\langle \cos \theta \rangle$ as the table carries it, which is 0 where nothing scatters and may be slightly negative in the far infrared.
- `albedo` (`m%NLAM`) (optional): $C_{\text{sca}}/C_{\text{ext}}$, and 0 where C_{ext} underflows to zero. It is derived here rather than by the caller so that every host gets the same convention at those wavelengths.
- `status` (optional): error report; see §4.5. It is 0 on success, 1 if an output array is not of size `m%NLAM`, 2 if no extinction table was loaded for this model, and 3 if `m%lam` reaches outside the table.

5.1 What the call returns, and where it comes from

`dust_extinction` reads the size-integrated curve from the precomputed table its builder loaded (`m%kext_path`; see the `kext_path` argument in §3. and Table 2). Nothing is integrated over grain size at call time. The size integral itself is the separate routine `size_integrated_extinction`, which has the same argument list, is what `calc_kext.x` calls, and is what wrote those tables:

$$\begin{aligned} C_{\text{abs}}(\lambda) &= \sum_{\text{pop}} \sum_a dn(a) C_{\text{abs}}(\lambda, a), \\ C_{\text{sca}}(\lambda) &= \sum_{\text{pop}} \sum_a dn(a) C_{\text{sca}}(\lambda, a), \\ \langle \cos \theta \rangle &= \frac{\sum_a dn C_{\text{sca}} g}{\sum_a dn C_{\text{sca}}}, \end{aligned}$$

a plain binned sum over every population, since $dn(a)$ already carries the bin width. It is exported too, for a host that would rather redo the integral than read the product; `dust_extinction` is the route that gives an RT host exactly the numbers that were checked against the HD23 release and Draine’s own `kext_albedo` tables (§6.5).

Why the builder holds the table. The table paths are relative to `sed/`, and a host typically changes directory back as soon as the model is built — so the file has to be opened while the builder still runs, not when `dust_extinction` is called. That is the whole reason `kext_path` is a builder argument.

Interpolation onto `m%lam`. Needed only where the table grid and the model grid differ. C_{ext} , C_{abs} and C_{sca} are positive and locally power-law in λ , so they are interpolated linearly in $(\ln \lambda, \ln C)$; $\langle \cos \theta \rangle$ changes sign and has no logarithm, so it is linear in $\ln \lambda$. Where a model wavelength *coincides* with a table wavelength the table value is returned unchanged, which is the normal case for the shipped defaults — the EUV table’s nodes are the plain grid’s nodes over the overlap. Outside the tabulated range there is no optics to serve, so the call is refused (`status = 3`) rather than extrapolated.

Two quantities are not read straight from the file. `albedo` is re-derived as $C_{\text{sca}}/C_{\text{ext}}$ from the interpolated cross sections instead of from the file’s own `albedo` column, because the cross sections are written with more digits (the oldest product rounds its `albedo` column to six decimals, which floors to 0 in the far infrared). `gbar` is taken from the table as it stands: it is already the scattering-weighted average, and re-weighting it would need the size-resolved optics the table does not carry.

Every model scatters. All four builders populate the extinction-side scattering optics, so the `albedo` and `gbar` in every table are physical:

- **astrodust:** C_{sca} and $\langle \cos \theta \rangle$ from the random-orientation T-matrix Q table (and, below $0.0912 \mu\text{m}$, from Mie on the same dielectric function).
- **DL07:** Mie on the D03 astrosilicate and graphite dielectric functions (`q_silicate_full / q_graphite_full`), on the same random-orientation average ($\frac{1}{3} E \parallel c + \frac{2}{3} E \perp c$) as the absorption. The two carbonaceous charge states share one scattering description: charge shifts PAH absorption features, not the graphite sphere’s scattering.

- **Zubko / file-defined:** the Q_{sca} and g columns of the model's own DustEM tables, i.e. the authors' own multilayer-sphere solution, read from the same file as Q_{abs} .

A population that genuinely does not scatter leaves its scattering optics unallocated and contributes zero. In the astro dust model the PAHs are exactly that case — PAH scattering is Rayleigh-negligible — so they enter extinction through absorption only, and the astro dust grains carry all the scattering and the asymmetry.

5.2 Dust mass per H, and mass opacity

Everything above is *per H nucleon*. A host that carries a dust mass density instead needs the model's own dust mass per H to convert:

```
Mdust_H = dust_mass_per_H(m) ! [g/H], one wavelength-independent constant
Kabs = Cabs / Mdust_H ! [cm^2/g], Draine's mass opacity
```

It is the model's own size distribution weighted by the solid density of each population's material,

$$\frac{M_{\text{dust}}}{N_{\text{H}}} = \sum_{\text{pop}} \rho_{\text{bulk}} \sum_a \frac{4}{3} \pi a^3 dn(a),$$

with a the effective radius in cm and $dn(a)$ the same binned number per H the size integral above uses, so no da enters the sum. The density lives on the population (`m%pop(ip)%rho_bulk`, §8.); a population whose model states none leaves it at 0 and contributes no mass. The function reads `m` only — it needs no radiation field and no built optics beyond the model itself.

Every builder states its densities, so the quantity is defined for every model in the tree:

model	grain densities [g/cm ³]	$M_{\text{dust}}/N_{\text{H}}$ [g/H]
astrodust	astrodust 2.74, PAH 2.0 (HD23)	1.185×10^{-26}
DL07	astrosilicate 3.5, graphite 2.2	1.750×10^{-26}
Zubko	each component's own DustEM header (3.5, 2.24, 2.24)	1.444×10^{-26}
file-defined	the pop: line's rho, else the optics file's	—

The astro dust numbers are Hensley & Draine (2023): $\rho_{\text{Ad}} = 2.74$ is the porosity-corrected solid density of the astro dust material, and $\rho_{\text{PAH}} = 2.0$ is the density behind that model's $N_{\text{C}} = 417(a/\text{nm})^3$. The DL07 numbers are stated by that model's own paper, Draine & Li (2007) §2, in the paragraph defining the effective radius $a = (3M/4\pi\rho)^{1/3}$: amorphous silicate 3.5 (also the density the DL01 astrosilicate enthalpy is built on), and carbonaceous grains “a mass density due to graphitic carbon alone of $\rho = 2.2 \text{ g cm}^{-3}$ ”. That is deliberately *not* the 2.24 of WD01 and LD01, which is the density implied by the carbon-atom count $N_{\text{C}} = 470(a/\text{nm})^3$ the DL07 builder selects; DL07 kept that count and quoted the rounder density, and the model being built here is DL07's. The DL07 carbonaceous grains are one material sequence, PAH-like at small a and graphite-like at large a with no size at which the solid changes, so the one density is used across the whole sequence. That the astro dust PAHs take 2.0 and the DL07 carbonaceous grains 2.2 is a difference between the two *models*, not a correction of one by the other. The Zubko densities are read out of the `Density` entry in the header of each component's own DustEM optics table.

Check against DL07, and a caution about his data files. Table 3 of Draine & Li (2007) lists $M_{\text{dust}}/M_{\text{H}}$ for each of its physical dust models. Model $j_M = 7$, MW3.1_60, is the size distribution this builder selects, and the paper gives 0.0104. Ours is 1.750×10^{-26} g/H, i.e. 0.01046 — agreement to 0.55%.

Draine’s `kext_albedo_WD_MW_3.1_60_D03.all` header, for the same model, instead states $M_{\text{dust}}/N_{\text{H}} = 1.870 \times 10^{-26}$ g/H, which is $M_{\text{dust}}/M_{\text{H}} = 0.0112$ and therefore 7.4% away from his own Table 3. The same file disagrees with the paper on the silicate normalization as well: the header says the WD01 abundances were reduced by 0.93, while the caption of the paper’s Fig. 11 says 0.92.

The position taken here is to follow the *file* for the optics and the *paper* for the mass. The optics are not in question — our C_{abs}/H agrees with the 2003 table to better than 0.5% from 100 to $10^4 \mu\text{m}$, where absorption is in the Rayleigh limit and therefore proportional to the grain volume, so the material volumes agree with his. What is left is the $\sim 1\%$ that the two normalizations differ by, and it is Draine’s own internal inconsistency, not ours. It is worth knowing that K_{abs} from any of these tables inherits whichever convention was used: C_{abs}/H is the better-determined quantity, and K_{abs} is that number divided by a convention.

6. The extreme-ultraviolet extension

The astro dust model’s wavelength grid is the T-matrix Q table’s, which stops at $0.0912 \mu\text{m}$ — 13.6 eV, the Lyman limit. A host that transports ionizing radiation needs shorter wavelengths, so both `build_astrodust` and `build_dl07` take an optional `lam_min` [μm]:

```
call build_astrodust(m, qtab, sizedist, NT, T_lo, T_hi, lam_min=1.001e-4_wp)
```

Log-spaced points are then prepended from `lam_min` up to the table, at a step no coarser than the table’s own $\Delta \ln \lambda = 0.01156$ at its short-wavelength end, with `m%lam(1)` set to `lam_min` exactly. **Omitting `lam_min` prepends nothing and leaves the model bit-identical to before.**

6.1 Where the optics come from in the extended band

- **Astro dust grains.** The T-matrix on the Draine & Hensley (2021) dielectric function — the same material *and* the same $b/a = 1.400$ oblate spheroid as the Q table, on the same random-orientation average, so the band continues the table’s own calculation instead of substituting a different particle for it. This is the default. Passing `euv_tmatrix=.false.` instead selects Bohren–Huffman Mie for the *volume-equivalent sphere*: much cheaper, but an approximation. Both are described in §6.2.
- **PAH / carbonaceous.** Above 21.4 eV ($1/\lambda = 17.25 \mu\text{m}^{-1}$) the DL07 PAH cross section is zero by construction, so graphite is the *whole* carbonaceous absorption there. Below that energy the D16 turbostratic-graphite sphere table is used as before; above it the code switches to Mie on the D03 graphite dielectric functions (random orientation, $\frac{1}{3}$ parallel + $\frac{2}{3}$ perpendicular), because the D16 table’s radius axis bottoms out at $10^{-3} \mu\text{m}$ while the astro dust size grid starts at $3.55 \times 10^{-4} \mu\text{m}$. In the Rayleigh limit $Q_{\text{abs}} \propto a$, so clamping a smaller grain to the table’s first radius overestimates its absorption by $10^{-3} \mu\text{m}/a$ — up to $2.4\times$ for the smallest PAH the size distribution

populates $(4.22 \times 10^{-4} \mu\text{m})$. *This branch is unreachable on the unextended grid*: its shortest wavelength gives $1/\lambda = 10.96 < 17.25$, so every previous result is unaffected.

- **DL07 model.** Its optics are Mie on the D03 dielectric functions at *every* wavelength, so nothing changes character across the band and the extension is a grid extension only.
- **Zubko model.** No extension, and none possible: its tabulated optics already span 10^{-3} to $10^4 \mu\text{m}$, and those tables *are* the model definition, so there is no dielectric function to extend them with.

6.2 The two routes, and what the cheap one costs

The Q table is a random-orientation T-matrix solution for an oblate spheroid of axial ratio $b/a = 1.4$. **By default the extended band is the same solution**, called directly (spheroid_q in tmatrix/driver/spheroid_optics.f90) on the same dielectric function, with the same convergence tolerance and the same regime bounds the table's own driver uses: the Rayleigh dipole limit below $x = 2\pi a_{\text{eff}}/\lambda = 0.1$, the T-matrix between 0.1 and 50, the geometric-optics limit above 50. Nothing about the particle changes across the seam.

That is not free. Measured on this machine (gfortran 13.1, -O3, 167 radii; the timings are the band alone, as sed_init reports them):

lam_min	n_{euv}	band points	of which T-matrix	threads	time
0.0247968 μm (50 eV)	113	18 871	12 204	1	527.5 s
”				2	264.2 s
”				4	136.7 s
”				8	69.2 s
$1.001 \times 10^{-4} \mu\text{m}$ (12.4 keV)	590	98 530	41 900	72	73.5 s

The second row is the floor `calc_kext.x_astrodust_euv` picks by default. Most of a deeper band is cheap, because past $x = 50$ it is the geometric-optics limit: going from 50 eV down to 12.4 keV multiplies the band points by 5.2 but the T-matrix points by only 3.4.

The band is built with OpenMP over grain radii, with one T-matrix workspace for each thread (37 MiB at allocation, 80 MiB once its full-direct arrays are in place; nothing is allocated when there is no band to compute). That is the one thing to watch on a many-core host: peak resident memory measured 64 MB / 110 MB / 197 MB / 376 MB at 1 / 2 / 4 / 8 threads and 3.0 GB at 72, so cap `OMP_NUM_THREADS` if memory is tight. Each radius writes its own column and reads no other, so the result does not depend on the thread count or the schedule — the four runs above wrote byte-identical optics.

The cheap route. `euv_tmatrix=.false.` replaces the spheroid by its volume-equivalent *sphere* and solves it with Bohren–Huffman Mie, in a tenth of a second. The justification for that substitution is **not** the anomalous-diffraction argument $|m-1| \ll 1$: for this material $|m-1| = 0.77$ at the seam and still 0.56 at 20 eV. It rests instead on the two limits that actually apply:

- **Small grains are in the Rayleigh limit** ($x = 2\pi a/\lambda = 0.033$ at the seam for the smallest populated radius, and only 0.24 at 100 eV). There the ellipsoid polarizability is $\alpha_j = V(\epsilon - 1)/\{4\pi[1 + L_j(\epsilon - 1)]\}$, so shape enters *only* through $L_j(\epsilon - 1)$. And $|\epsilon - 1|$ falls from 1.86 at 13.5 eV to 1.10 at 20 eV, 0.28 at 40 eV and 0.061 at 100 eV: the depolarization factors drop out and sphere and spheroid converge on the same C_{abs} .
- **Large grains are in the geometric-optics limit**, where $C_{\text{ext}} \rightarrow 2 \times (\text{orientation-averaged projected area})$. By Cauchy's theorem that average is $S/4$, and for $b/a = 1.4$ the spheroid's $S/4$ exceeds πa_{eff}^2 of its volume-equivalent sphere by 2.12%. Mie is therefore low by $1 - 1/1.0212 = 2.08\%$ in that limit. **The error converges to a constant, not to zero**, however far into the EUV one goes.

Consequently the error does *not* fall monotonically with photon energy. Only $a \lesssim 2 \times 10^{-3} \mu\text{m}$ improves monotonically; $a \gtrsim 0.01 \mu\text{m}$ gets *worse* between 13.6 and 30 eV before recovering, and the largest sizes settle at a nearly energy-independent -1.9 to -2.0% . Over the whole band and the whole size range the error stays within about 2%; `sed/src/q_astrodust.f90` tabulates it size by size.

Measured against the default. Building the same model both ways with `lam_min = 0.0247968 μm` gives, for $\Delta C_{\text{abs}} = (\text{Mie} - \text{T matrix})/\text{T matrix} [\%]$:

$a [\mu\text{m}]$	13.8 eV	17.7	24.8	30.1	40.2	50
4.73×10^{-4}	-0.31	+0.71	-0.01	-0.08	-0.13	-0.02
1.00×10^{-3}	-0.14	+0.75	+0.05	-0.00	-0.03	-0.02
5.01×10^{-3}	-0.16	+0.75	-0.09	-0.17	-0.24	-0.23
1.00×10^{-2}	+0.30	+0.24	-0.60	-0.57	-0.37	-0.27
5.01×10^{-2}	-1.28	-1.47	-1.59	-1.58	-1.47	-1.27
1.00×10^{-1}	-1.62	-1.74	-1.85	-1.87	-1.86	-1.77
2.99×10^{-1}	-1.90	-1.96	-2.01	-2.02	(geometric optics)	

Over the $0.1 \leq x \leq 50$ block where the T-matrix itself answers, ΔC_{abs} runs from -2.03 to $+0.77\%$ with a median of -0.70% , exactly the bound argued above. Where $x > 50$ both routes leave the T-matrix — the spheroid one for its geometric-optics limit, which is what the Q table uses there too — and the two disagree by -6 to -19% ; that is Mie against a chord approximation, not against the T-matrix.

Seam continuity. Extrapolating the Q table's first two wavelengths log-log back to the last extended-band wavelength and comparing with what the band actually produced there, over all 167 radii:

route	median step	max step	RMS step
T-matrix (default)	0.012%	0.068%	0.027%
Mie sphere (<code>euv_tmatrix=.false.</code>)	1.167%	15.709%	6.186%

The default is continuous to a few hundredths of a percent, about a hundred times smoother than the sphere substitution. The asymmetry parameter behaves the same way: the table gives $\langle \cos \theta \rangle = 0$ wherever its own regime bounds put a

radius in the Rayleigh or geometric-optics limit, and the default reproduces that exactly, whereas the sphere route steps from 0.90 to 0 across the seam for $a \gtrsim 1 \mu\text{m}$.

The shipped extinction table. `data/kext_astrodust_MW_euv.dat` is integrated with the default T-matrix route, so the extinction it serves and the emission the model computes are one calculation of one particle at every wavelength. It was regenerated for that change; the 1129 rows at $\lambda \geq 0.0912 \mu\text{m}$ are byte-identical to the sphere-route version, because those come from the Q table either way. Below the seam the sphere route was low in C_{abs} and K_{abs} by up to 3.6%, high in C_{sca} by up to 4.6% and in albedo by up to 5.1%, and wrong in $\langle \cos \theta \rangle$ by up to a factor of four at the shortest wavelengths, where the sphere keeps a forward-scattering asymmetry the spheroid's geometric-optics limit does not. C_{ext} moved least, by 0.72%. Regenerating costs about five minutes on 64 threads (OMP_NUM_THREADS=64 `./calc_kext.x astrodust euv`); pass `euv_tmatrix=.false.` to build_astrodust first if the sphere route is wanted instead.

6.3 The single-photon ceiling comes from the field

The stochastic solvers set the top of a grain's enthalpy window from the energy of the hardest single photon it can absorb. That ceiling used to be the hard-coded Lyman limit, 13.6 eV, which an ionizing band makes wrong: a photon harder than the ceiling drives an excursion the window cannot hold. It is now

$$\max(13.6 \text{ eV}, hc/\lambda_{\text{hard}}), \quad \lambda_{\text{hard}} = \min \{ \lambda_i : J_{\lambda}(\lambda_i) > 10^{-12} \max_j J_{\lambda}(\lambda_j) \},$$

computed by `hardest_photon_energy` in `sed/src/radfield.f90` and called from all three places that need it — `sed_grain_loop` and `narrow_iterative` in `sed_astrodust.f90`, and `qm_solve_grain` in `stoch_qm.f90`. Any units may be passed for J_{λ} , since only ratios within the array are used.

The ceiling is a property of the field, not of the grid. That distinction is the whole point, and a coincidence hides it. A model's wavelength grid is the grid of its optics tables, which can reach far past the band a transported field occupies:

model	grid floor	implied photon energy
astrodust	0.0912 μm (T-matrix Q table)	13.595 eV
DL07	0.0912 μm (the same table)	13.595 eV
Zubko	$1.000 \times 10^{-3} \mu\text{m}$ (DustEM tables)	1239.8 eV

For astrodust and DL07 the grid floor *is* the Lyman limit, so $13.595 \text{ eV} < 13.6 \text{ eV}$: the maximum returns the constant, and grid and field give the same answer whichever one is measured. For Zubko they differ by a factor of 91, and the field is the one that is right — a Mathis field carries no photon at all below 0.0912 μm , however far the optics table extends.

Taking it from the grid costs resolution, not physics. `umax` sets the upper edge of a *fixed* number of enthalpy bins, so a ceiling 91 times too high starts the solve with bins 91 times too coarse. The refinement loops do contract the window once the top bin is unpopulated, but the contraction is guarded (`umax > 1.02*umaxlo` and `umax > 1.01*ub(jcut)`)

and capped at $\text{MAX_ITER} = 10$, so it does not return to where it began; expansion carries no such guard. Measured in a host code on the Zubko model, with no lam_min requested at all: dust temperature $+0.0705\text{ K}$ median and positive in every cell, the ten brightest SED bins -0.9 to -1.9% , the emission image 1.9% median and 5.4% maximum, total emitted energy -0.08% . A systematic redistribution at nearly conserved energy is what a coarser bin grid produces, and docs/EUV_EXTENSION_HOST_REGRESSION.md records the measurement.

Why 10^{-12} of the peak, and not simply $J_\lambda > 0$. Exact zeros must be excluded — J_{Mathis} returns exactly zero below $0.0912\mu\text{m}$ — but an exact positivity test is too fragile: a host that hands over denormal or roundoff-level residue in its unilluminated bins would push the ceiling back up by orders of magnitude. A component a fraction 10^{-12} below the peak of J_λ contributes at most that fraction of the photon absorption rate; allowing two decades for the run of C_{abs} across the illuminated band, the enthalpy states it could populate carry probability $\lesssim 10^{-10}$ of the peak, at or below the 10^{-13} tail level (PMIN_UP , PMIN_UP_QM) at which both solvers already truncate the window. Such a component cannot change a resolved excursion, and 10^{-12} still sits eighteen decades above the residue it is meant to reject.

The error is one-sided on purpose. Both refinement loops *expand* the window while the top bin is still populated, so an underestimated ceiling is corrected as the loop runs; the guarded contraction can stop short of where it began, so an overestimate is not corrected at all. $\text{MAX_ITER} = 10$ bounds the correction in either direction. Overestimating is therefore the dangerous direction, which is why the threshold is set high enough to reject junk rather than as low as floating point allows.

main_zubko.x exists to keep this path covered. The defect escaped main_astrodust.x and main_dl07.x because for those two models the grid floor and the field's short end are the same number: no driver in the tree ran the emission solvers on a model whose optics grid reaches past the illuminated band. main_zubko.x (§7.4) closes that gap and is built by the default make. It prints the two candidates side by side, so the difference shows in the first lines of output:

```
$ ./main_zubko.x # Mathis field, stops at the Lyman limit
optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
hardest photon in the field: 1.35946E+01 eV
single-photon bound in use : 1.36000E+01 eV
$ ./main_zubko.x euv # a hard component added below the Lyman limit
optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
hardest photon in the field: 3.70141E+02 eV
single-photon bound in use : 3.70141E+02 eV
```

Without euv the ceiling must stay at 13.6 eV however far the optics grid extends, and the Zubko SED is then identical to what the hard-coded constant produced; with euv it must follow the field up, and it does.

A model built without lam_min is unaffected. For astro dust and DL07 the maximum selects the 13.6 eV constant on the unextended grid either way, so those results are unchanged to the last bit.

6.4 What else the extension does, and does not, change

- **The Planck integrals are anchored to the table range.** The internal $\log\lambda$ integration grid for $\kappa_B(T, a)$ keeps its step and its sample points over the optics-table range and merely *prepends* points below it, so widening the model grid cannot coarsen the sampling of the range that carries the integrand. Measured: with a $1.001 \times 10^{-4} \mu\text{m}$ floor, κ_B is unchanged to round-off over the thermal range ($\max |\Delta\kappa_B|/\kappa_B = 3.4 \times 10^{-16}$ for $T \leq 3000 \text{ K}$, and exactly zero below $\sim 2400 \text{ K}$); κ_{CMB} is bit-identical. The only nonzero difference is 1.1×10^{-8} at the very top of a 5000 K grid, where the Wien tail genuinely reaches into the extended band — that is real extra emission, not a sampling artifact. Without the anchoring, spending a fixed budget of points on the widened interval would have moved κ_B by $\sim 10^{-4}$ for a purely numerical reason.
- **Stochastic heating by hard photons is not solved by this change.** The extension fixes the photon-energy mapping and the absorbed-energy accounting. It does not address the finite enthalpy grid: upward transitions above the top of that grid are accumulated into the highest temperature bin, and for the smallest PAHs a 54 eV photon already exceeds $H(5000 \text{ K})$. Nor does it add photoionization, electron energy loss, fragmentation or PAH destruction. Treat the extended band as extinction and absorbed-energy physics, not as a validated hard-EUV PAH *emission* model.

6.5 How far each model can be checked

author-provided data	coverage	what it contains
Draine, kext_albedo_WD_MW_3.1_60_D03 (DL07 / WD01)	10^{-4} – $10^4 \mu\text{m}$ $= 1.24 \times 10^{-4} \text{ eV}$ – 12.4 keV	λ , albedo, $\langle \cos\theta \rangle$, C_{ext}/H , K_{abs} , $\langle \cos^2\theta \rangle$
HD23 astrodust release (extinction.dat, scattering.dat, wav.dat)	0.1 – $3 \times 10^4 \mu\text{m}$ $= 4.1 \times 10^{-5}$ – 12.4 eV	absorption and scattering τ/N_H only. No $\langle \cos\theta \rangle$ product exists anywhere in the release.
Zubko ZDA optics tables	10^{-3} – $10^4 \mu\text{m}$ $= 1.24 \times 10^{-4} \text{ eV}$ – 1.24 keV	Q_{abs} , Q_{sca} , Q_{ext} , g for each radius

Table 4: What the model authors published, and how far it reaches.

The consequence is blunt: **the astrodust EUV band has no external reference at all**, because the HD23 release stops at 12.4 eV — exactly where the extension begins. What can be said is that the band is computed the same way the published part of the model is, from the same dielectric function and for the same particle (§6.2), and that it joins the table without a step. That is a consistency argument, not a validation.

The DL07 model *can* be checked, because Draine published the same model over the whole range. Comparing data/kext_dl07_MW_euv.dat with kext_albedo_WD_MW_3.1_60_D03.all_2003 on the reference’s own wavelengths (and counting only points where the reference grid is no finer than ours — elsewhere the comparison measures our grid, not our optics) gives, per decade in λ , mean $|\Delta C_{\text{ext}}|/C_{\text{ext}}$ of 0.056–0.86%, mean $|\Delta \text{albedo}| \leq 0.0019$ and mean $|\Delta \langle \cos\theta \rangle| \leq 0.00054$. calc_kext.x dl07_euv prints that table. The two residual features are worth naming: the

excluded points cluster at the X-ray absorption edges (Fe K, Si K, Mg K, Fe L, O K, C K), which the reference brackets far more finely than our $\Delta \ln \lambda = 0.0116$ log grid can resolve; and the largest single deviation, 7.9% in the 1–10 μm decade, is confined to the 6.2, 7.6 and 7.9 μm PAH C–C features, where Draine’s 2003 table uses a different vintage of the PAH cross sections.

7. Standalone programs

make in SEDust/sed/ builds five executables. All of them are run *from* SEDust/sed/, because every data path is relative to ../.

7.1 calc_kext.x — size-integrated transport optics

Builds a model and calls size_integrated_extinction on it, then writes the result under ../data/. Every model goes through the same two calls, and dust_extinction later serves these very files back to an RT host, so the files written here *are* the optics that host gets in memory.

```
./calc_kext.x astrodust [euv | <lam_min in um>]
./calc_kext.x dl07 [euv | <lam_min in um>]
./calc_kext.x zubko [formula | table]
./calc_kext.x from_files <descriptor> [data_dir]
```

With no argument it prints its usage and stops. euv carries the grid down to whatever the model’s own dielectric function covers; an explicit number requests a specific floor in μm . zubko defaults to formula. from_files takes the descriptor path, and the data directory defaults to the descriptor’s own directory.

Columns are

$\lambda[\mu\text{m}]$ albedo $\langle \cos \theta \rangle$ C_{ext}/H C_{abs}/H C_{sca}/H K_{abs}

in μm and cm^2 per H nucleon, with K_{abs} in cm^2/g . The first four columns are in the order of Draine’s kext_albedo tables, so a reader written for those files (for instance the par%ion_dust_kext reader of the MoCHII host, which takes four reals from each row and uses λ , albedo and C_{ext}) parses these unchanged. Note the fifth column differs from Draine’s: his is K_{abs} [cm^2/g], ours is C_{abs}/H [cm^2/H], so no dust-mass normalization is folded into the first six columns. $K_{\text{abs}} = (C_{\text{abs}}/H)/(M_{\text{dust}}/N_{\text{H}})$ is written as a separate trailing column instead, on every product, because every model states its grain densities (§5.2). The header of each file records the $M_{\text{dust}}/N_{\text{H}}$ its K_{abs} column is normalized by and where that model’s densities come from.

After writing, the program checks $C_{\text{ext}} = C_{\text{abs}} + C_{\text{sca}}$ and the ranges of albedo and $\langle \cos \theta \rangle$, and — for astrodust and DL07 — prints the comparison against the author-provided data of Table 4.

These files are transport optics, not a dust model. They carry no size-resolved $C_{\text{abs}}(\lambda, a)$, no grain enthalpy and no Planck integral, so stochastic heating and thermal emission cannot be computed from them. A host that needs emission must build the model; the builder reads this curve back in, dust_extinction serves it on the model wavelength grid

and `dust_emission` supplies the re-emitted spectrum, which is what keeps the absorbed power and the emission referring to one and the same grain.

7.2 `main_astrodust.x` — the `astrodust` production SED

Solves the `astrodust+PAH` SED for a single Mathis-ISRF cell at $U = 1.585$ ($\log U = 0.20$, the HD23 best fit), on a 200-point $\log T$ grid from 2.7 to 5000 K. Reads the T-matrix Q table `../tmatrix/output/q_astrodust_P0.20_Fe0.00_1.400.dat` and `../data/release/size_distribution.dat`; writes `output/astrodust_irem_ours_<tag>S1.dat`, `... S2.dat` and `... PAH.dat`, each with λ [μm] and $\lambda I_\lambda/N_H$. Optional arguments, in any order, each of which tags the output filename so a non-default run cannot clobber the production one: `qm`, `qm_dbcon`, `draine` (solver); `logU=X`; `nstate=N`, `nisrf=N` (QM solver sizes); `induced` (turn the stimulated-emission factor on); `photcut`; `c2` (Stage-1 density-corrected prefactor); `mathis_orig`.

The tag is assembled after all the arguments have been read, in one fixed order — solver, enthalpy stage, radiation field, emission term, numerical settings — so the order they are typed in does not matter: `induced qm` and `qm induced` both write `... _qm_induced_S1.dat`. The three solver options select the same thing, so naming two of them is refused rather than resolved by taking the last. The driver prints the filename it will write before it starts solving.

7.3 `main_dl07.x` — the `DL07` reference SED

Solves the Draine & Li (2007) silicate + carbonaceous SED at $U = 1$ with the WD01 size distribution. `./main_dl07.x [model] [qm|draine]`, where `model` is one of `mw31_00` `... mw31_60` (default `mw31_60`), `lmc2_00`, `lmc2_05`, `lmc2_10`, `smc`; an unknown name is rejected with the valid list. Writes `output/dl07_sed_ours_<model>[_qm|_draine].dat` with columns λ , total, silicate and carbonaceous $\lambda I_\lambda/N_H$.

7.4 `main_zubko.x` — the `Zubko/ZDA` SED, and the wide-grid regression

Solves the Zubko, Dwek & Arendt (2004) BARE-GR-S SED — PAH + graphite + silicate, the ZDA size-distribution formula, the DustEM optics and calorimetry tables — for a Mathis ISRF at $U = 1$, and writes `output/zubko_sed_ours[_<solver>][_euv].dat` with columns λ , total, PAH, graphite and silicate $\lambda I_\lambda/N_H$.

```
./main_zubko.x [heuristic | draine | qm | equil] [euv]
```

The solver defaults to `heuristic`, the model default. `euv` replaces the Mathis field below the Lyman limit by a diluted 10^5 K blackbody, so that the field really does carry photons above 13.6 eV; the dilution is chosen so that band deposits roughly as much power as the whole Mathis field — enough for the hard photons to matter, little enough that the small grains still heat stochastically.

This is the emission counterpart of `./calc_kext.x zubko`, which exercises only the extinction size integral. It exists because Zubko is the one shipped model whose optics grid ($10^{-3} \mu\text{m}$) reaches far past the band a transported field occupies ($0.0912 \mu\text{m}$ for the Mathis ISRF), so anything in the stochastic-heating path that reads the grid where it should read the field shows up here and nowhere else. Running it with and without `euv` is the regression of §6.3.

7.5 make_enthalpy.x — enthalpy tables (diagnostic)

Tabulates $U(T, a_{\text{eff}})$ for the two astrodust enthalpy stages and writes output/enthalpy_S1.dat and output/enthalpy_S2.dat (201 log-spaced temperatures from 2.7 to 5000 K $\times$ the 169 radii of ../data/dielectric/DH21_aeff, T_{outer} and a_{inner}). It takes no arguments. **It is not required to compute an SED:** the solver calls the closed-form Debye sums directly in its inner loop. Use it to compare the two stages, to plot $U(T)$, or to spot-check against published DL01 values.

8. The model object and accessors

```
type :: dust_model_t
  character(len=32) :: name
  integer :: NA, NLAM, NT
  real(wp), allocatable :: lam(:) ! (NLAM) [um]
  real(wp), allocatable :: aeff(:) ! (NA) [um]
  real(wp), allocatable :: T_first(:) ! (NT) [K]
  real(wp), allocatable :: log_T_first(:) ! (NT)
  type(grain_pop_t), allocatable :: pops(:)
  integer :: n_channel
  character(len=16), allocatable :: channel_name(:)
  logical :: use_induced_emission ! default .false.
  character(len=16) :: stoch_method ! default 'heuristic'
  logical :: verbose ! default .false.
end type
```

Each grain_pop_t in m%pops carries its own aeff, dn, Cabs, Csca, gsca, kappB, H and kappCMB (plus cached logs), and the scalar rho_bulk [g/cm³], the solid density of the material that population is made of. A model whose components have different size grids — Zubko is the case in the tree — is represented by populations of different lengths, which is why the solver takes the size count from size(pops(ip)%dn) rather than from m%NA.

rho_bulk. It is what turns dn into a mass, and the only thing dust_mass_per_H (§5.2) reads besides the size distribution. For a porous grain it is the density already reduced by the porosity, i.e. mass over the *grain* volume $\frac{4}{3}\pi a_{\text{eff}}^3$, not over the solid volume inside it — which is why the astrodust value is $\rho_{\text{Ad}} = 2.74$ rather than the 3.425 of the solid it is made of. Leaving it at its default 0 declares that the model states no density, and that population then contributes no mass.

Diagnostics. m%verbose (default .false.) keeps the library solve path silent; set it .true. to print the same solver diagnostics as the CLI drivers.

Convenience accessors (all in dust_lib):

```
n = dust_nlam(m) ! number of wavelengths
```

```
nc = dust_n_channel(m) ! number of output channels
lam = dust_lambda(m) ! allocatable copy of the wavelength grid [um]
nm = dust_channel_name(m, ic) ! name of channel ic
```

9. The generic file loader

`build_from_files` reads a small text *descriptor* and assembles a model from data files, with no code changes. One pop: line per population:

```
# data/zubko/zubko_descriptor.txt
name = zubko_bare_gr_s
# pop: <grain_type> <channel> <optics> <dnda_table> <calorimetry> <rho(0=use file)>
pop: pah 1 PAH_28_1201_neu.dat ZDA_BARE_GR_S_SzDist_PAH.dat Graphitic_Calorimetry_1000.dat 0.0
pop: gra 2 Gra_121_1201.dat ZDA_BARE_GR_S_SzDist_Gra.dat Graphitic_Calorimetry_1000.dat 0.0
pop: sil 3 suvSil_121_1201.dat ZDA_BARE_GR_S_SzDist_Sil.dat Silicate_Calorimetry_1000.dat 0.0
```

- grain_type: 'sil', 'pah', or 'gra' (selects the heat-capacity/mode model used by the 'qm' solver; ignored by the GD solvers, which use the supplied enthalpy directly).
- channel: integer output channel (1...); populations sharing a channel are summed.
- optics: a DustEM/Zubko Q -table (Q_{abs} per radius and wavelength); $C_{\text{abs}} = Q_{\text{abs}} \pi a^2$.
- dnda_table: a 2-column table a [μm] and dn/da [$\text{cm}^{-1} \text{H}^{-1}$].
- calorimetry: a specific-enthalpy table $u(T)$ [erg/g]; $H(T, a) = u(T) \rho \frac{4\pi}{3} a^3$.
- rho: grain density [g cm^{-3}]; 0.0 uses the value in the optics-file header.

All files are sought under `data_dir`. The coded builders (`build_astrodust/dl07/zubko`) are the recommended path for the built-in models; `build_from_files` is for adding new data-defined models.

10. Linking into a 3D RT code

Compile your RT source against *both* archives, with `.mod` search paths pointing at `SEDust/sed/` and `SEDust/tmatrix/`:

```
gfortran -I<sed> -I<tmatrix> my_rt.f90 \
        -L<sed> -Lsedust -L<tmatrix> -ltmatrix -fopenmp -o my_rt.x
```

`libtmatrix.a` entered the contract when the `astrodust` EUV band moved to the T-matrix (§6.2); a host that links `-lsedust` alone now fails at link time. The archive itself needs no OpenMP runtime — the threading is on the `SEDust` side — so `-fopenmp` is required only because `libsedust.a` uses it.

A complete minimal example is provided in `SEDust/sed/rt_example/use_dustlib.f90`: it builds a model, computes emission for one field, and prints the SED peak. No `ISO_C_BINDING` is needed — the RT code simply uses the Fortran module `dust_lib`.

Threading. `m` is read-only during `dust_emission`; all scratch is local. The solver uses OpenMP internally over grains. If your RT parallelizes over cells, take care to avoid oversubscription (nested parallelism); the default 'heuristic' solver is serial per call, which suits an outer cell-level parallel loop.

11. Units and conventions

- Wavelengths λ_m are in μm throughout; cross sections in cm^2 .
- J_λ is a mean intensity with the same units as the Planck function B_λ (specific intensity per steradian). The library's output $\lambda I_\lambda / N_H$ is then in $\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1} \text{H}^{-1}$.
- The CMB (2.725 K) is included in the heating field by `J_Mathis`.
- The induced-/stimulated-emission factor is off by default (the output is *net* emission, matching the HD23 and DL07spec released SEDs).

12. Data files

Model data lives under `SEDust/data/`:

- `astrodust / DL07`: the T-matrix Q -table (`../tmatrix/output/q_astrodust_*.dat`) and the HD23 release size distribution (`../data/release/size_distribution.dat`).
- dielectric functions: `../data/dielectric/` — the DH21 `astrodust` index `index_DH21Ad_P0.20_0.00_1.400` (which supplies the `astrodust` EUV band), the D03 `astrosilicate` and `graphite` indices, and the D16 `turbostratic-graphite` sphere table.
- `Zubko`: `../data/zubko/` — the ZDA `config/formula`, the tabulated size distributions, the DustEM Q -tables, and the calorimetry tables from the Camps et al. 2015 benchmark; a ready descriptor is `../data/zubko/zubko_descriptor.txt`.

12.1 Transport-optics products

`calc_kext.x` (§7.) writes these into `SEDust/data/`. The last column is the point that matters when choosing one: **nothing here is a free choice — each floor is fixed by the data the model is made of**, and a shorter floor is refused rather than served with a refractive index frozen at its table boundary.

file	rows	λ range [μm]	what sets the short-wavelength floor
kext_astrodust_MW.dat	1129	0.0912–39810	the T-matrix Q -table grid (no extension requested)
kext_astrodust_MW_euv.dat [†]	1719	1.001×10^{-4} –39810	the DH21 astrodust dielectric function, whose own table stops at 1.00003×10^{-4}
kext_dl07_MW.dat	1129	0.0912–39810	the T-matrix Q -table grid (no extension requested)
kext_dl07_MW_euv.dat [†]	1761	6.205×10^{-5} –39810	the D03 dielectric functions; the shorter-reaching of astrosilicate and graphite stops at 6.1992×10^{-5}
kext_zubko_BARE_GR_S.dat [†]	1201	10^{-3} – 10^4	the ZDA optics table itself; this model has no extension and needs none

Table 5: Size-integrated transport optics written by `calc_kext.x`. All carry the columns of §7.. [†] marks the file its builder loads by default (Table 2); the others are there for a host that asks for them through `kext_path`. A non-EUV product is a row subset of its EUV counterpart — same grid, same numbers, over the overlap. `kext_astrodust_MW.dat` is a tracked regression reference and keeps its original narrower field widths; it is reproduced byte for byte by `./calc_kext.x astrodust`.